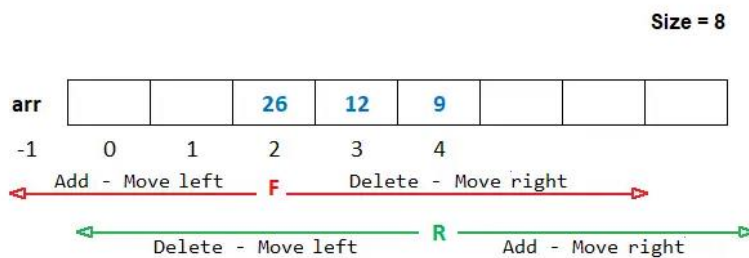


Double Ended Queue

Double Ended Queue in C++, also known as Deque, is a queue data structure in which insertion and deletion can be done from both left and right ends.



From the above image of the deque, we can see that when we add an element from the rear end, the R moves towards the right and, when we delete an element from the rear end, the R moves towards the left.

Similarly, when we delete an element from the front end, the F moves towards the right and when we add an element from the front end, the F moves towards the left.

Operation on Double Ended Queue

There are four operations possible on the double ended queue.

- Add Rear When we add an element from the rear end.
- Delete Rear When we delete an element from the rear end.
- Add Front When we add an element from the front end.
- Delete Front When we delete an element from the front end.

Add Rear Operation in Deque

For adding an element from the rear end first, we check if the value of `te` is equal to the size of the array then, we will display a message Queue is full, else we will increase the value of `R` by $R=(R+1)\%Size$ and add the element in the array at the new location of `R` and then increased the value of `te` by 1.

Here, `te` is a variable that keeps the total number of elements present in the array. When an element is added, the value of `te` is increased by 1. When an element has deleted, the value of `te` is decreased by 1.

```
if(te==size)
{
    cout<<"Queue is full\n";
}
else
{
    R=(R+1)%size;
    arr[R]=new_item;
    te=te+1;
}
```

Delete Rear Operation in Deque

For deleting an element from the rear end first, we check if the value of `te` is equal to 0 then, we will display a message Queue is empty, else we will check if the value of `R` is -1, then we will initialize the value of $R=size-1$. After that, we will display the deleted element on the screen and decrease the value of `R` and `te` by 1.

```
if(te==0)
{
    cout<<"Queue is empty\n";
}
```

```

else
{
    if(R==0)
    {
        R=size-1;
    }
    cout<<"Number Deleted From Rear End = "<<arr[R];
    R=R-1;
    te=te-1;
}

```

Add Front Operation in Deque

For adding an element from the front end first, we check if the value of te is equal to the size of the array then, we will display a message Queue is full, else we will check if the value of F is 0 then we will initialize the value of F=size-1, else decrease the value of F by 1. After that, add the element in the array at the new location of F and then increased the value of te by 1.

```

if(te==size)
{
    cout<<"Queue is full";
}
else
{
    if(F==0)
    {
        F=size-1;
    }
    else
    {
        F=F-1;
    }
    arr[F]=new_item;
    te++;
}

```

```
te=te+1;  
}
```

Delete Front Operation in Deque

For deleting an element from the front end first, we check if the value of te is equal to 0 then, we will display a message Queue is empty, else we will display the deleted element on the screen and then we will increase the value of F by $F=(F+1)\%Size$ and then decrease the value of te by 1.

```
if(te==0)  
{  
    cout<<"Queue is empty";  
}  
else  
{  
    cout<<"Number Deleted From Front End = "<<arr[F];  
    F=(F+1)%size;  
    te=te-1;  
}
```